

Outputs:

```
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterSynchronized
Final counter (synchronized + join) = 10000
Expected: 10000
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterJoinAfterStart
Final counter (join-after-start) = 10000
Expected: 10000
```

- Questions 2a and 2b, which do the same thing just differently, both show the correct answer

```
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterNoJoin
Final counter (no join) = 9000
Expected if we had waited: 10000
Run this several times - the number will vary.
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterNoJoin
Final counter (no join) = 1969
Expected if we had waited: 10000
Run this several times - the number will vary.
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterNoJoin
Final counter (no join) = 1802
Expected if we had waited: 10000
Run this several times - the number will vary.
```

- Question 2c, the program does not wait, and it almost always prints a value less than 10,000

```
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterNotSynchronized
Final counter (NOT synchronized, 1000/thread) = 10000
Expected: 10000

Final counter (NOT synchronized, 1,000,000/thread) = 2899613
Expected: 10000000
-> This one will almost always be LESS than expected. That's the race.
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java CounterNotSynchronized
Final counter (NOT synchronized, 1000/thread) = 9787
Expected: 10000

Final counter (NOT synchronized, 1,000,000/thread) = 3336435
Expected: 10000000
```

- Question 2d, the program has all workers working together but there is a clear race condition with the count

```

(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 100
n = 100
  sequential : sum = 348029, time = 3,666 ns (0.004 ms)
  2 threads  : sum = 348029, time = 117,084 ns (0.117 ms)
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 1000
n = 1,000
  sequential : sum = 3474108, time = 19,458 ns (0.019 ms)
  2 threads  : sum = 3474108, time = 91,333 ns (0.091 ms)
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 10000
n = 10,000
  sequential : sum = 34466466, time = 207,292 ns (0.207 ms)
  2 threads  : sum = 34466466, time = 207,750 ns (0.208 ms)
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 100000
n = 100,000
  sequential : sum = 340433609, time = 1,779,333 ns (1.779 ms)
  2 threads  : sum = 340433609, time = 1,158,833 ns (1.159 ms)
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 1000000
n = 1,000,000
  sequential : sum = 3389583622, time = 3,301,250 ns (3.301 ms)
  2 threads  : sum = 3389583622, time = 3,715,708 ns (3.716 ms)
(base) yuvalpaz@Yuvals-MacBook-Air lab14 % java -Xmx6g SumOfSquares 100000000
n = 100,000,000
  sequential : sum = 338391899136, time = 59,604,791 ns (59.605 ms)
  2 threads  : sum = 338391899136, time = 31,222,917 ns (31.223 ms)

```

- Question 3: As the number increases, the sequential time becomes less and less useful while the threads become more and more useful.